Five9
The Intelligent Cloud Contact Center

Messaging REST API

Developer's Guide

February 2025

This content describes the API requests and webhook notification events for the Messaging REST API.

About Five9

Five9 is the leading provider of cloud contact center software, bringing the power of the cloud to thousands of customers and facilitating more than three billion customer interactions annually. Since 2001, Five9 has led the cloud revolution in contact centers, delivering software to help organizations of every size transition from premise-based software to the cloud. With its extensive expertise, technology, and ecosystem of partners, Five9 delivers secure, reliable, scalable cloud contact center software to help businesses create exceptional customer experiences, increase agent productivity and deliver tangible results. For more information visit www.five9.com.

Trademarks

Contents

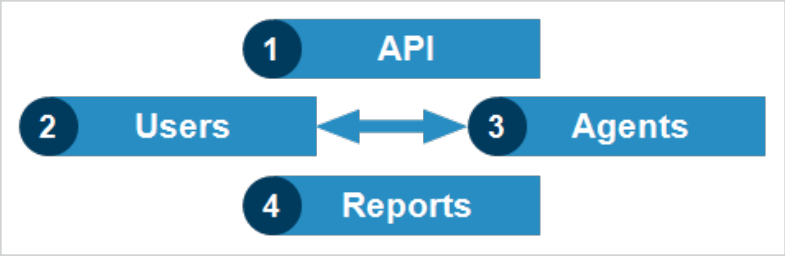
What's New

Release	Change	Topic
Feb 2025	• Updated GET call to include tokenID instead of SessionID. • Updated note about default expiration time from four to eight hours.	**Miscellaneous Issues** **Creates a Session for a Non-Agent User**
Nov 2024	Added Using Logged In Profiles for Availability Check.	**Miscellaneous Issues**
Aug 2024	Added a sub section to Syntax about initiating the login. Changed the URLs (from data center to common base) for the `Base_API_URL`element description.	Constructing Requests
Jan 2024	Revision to webhook timeout/retry.	**Notification Events**
Oct 2023	Added `attachments` to the Message and Conversation Message Event section.	**Message** **ConversationMessageEvent**
Jul 2023	Updated Create an Anonymous Session section.	**Creates a Session for a Non-Agent User**
May 2022	Updated URLs	**Constructing Requests**
Apr 2022	Updated Troubleshooting section.	**Events and Notifications**
Jul 2021	Updated description of Conversation data type.	**useBusinessHours**
Aug 2020	Added that the terms *tenant* and *domain* are used analogously.	**About the Five9 Messaging API**
	Added that the parameter `campaignID` is deprecated and users should use `campaignName` instead.	**Conversation**
Jul 2020	Initial release.	

About the Five9 Messaging API

The Five9 Messaging API enables Five9 agents who use Agent Desktop Plus or Five9 Plus Adapters for CRM applications to receive message interactions from third-party messaging platforms, such as SMS, WhatsApp, Facebook Messenger, WeChat, and RCS. You can integrate the Messaging API in your social messages channels, such as Facebook, Twitter, and WhatsApp. You can use the Messaging API to take advantage of all the Five9 Chat features, such as auto closing, requeuing, supervisor monitoring, generating digital engagement reports on chats, viewing metrics of agents engaging in chats, and sending comfort, welcome, business hours, and goodbye messages. The Messaging API creates Five9 conversations and processes them in Five9 Agent Desktop Plus or Five9 Plus Adapters.

Agents who have permission can process chat messages. Agents cannot initiate conversations but have access to chat transcripts and contact information. The chats created with the Messaging API are available in agent reports, such as the chat log report.



1. API
   - Conversion methods
   - Webhook events
2. Users
   - Send SMS or messages from social channels: Twitter, Facebook, and others
3. Agents
   - Receive and process messages in Agent Desktop Plus, CRM Plus adaptors, and Agent Desktop Toolkit Plus
4. Reports
   - Types of interactions in agent reports, such as Chat Log

Conversations are initiated by your customers who send messages to social network contacts or SMS numbers by using applications such as Twitter direct messaging. For more information about using this API, contact your Five9 representative.

> **Note**
> The terms *tenant* and *domain* are used analogously in this document.

Requirements

The Messaging API requires you to meet the following requirements:

- Your domain must be enabled for Five9 Engagement Workflow.
- You must have licenses for the following applications:
  - Five9 Agent Plus applications (Agent Desktop Plus or Five9 Plus Adapters for CRM).
  - Five9 Chat.

For more information, contact Five9 Support.

Limitations

Limitations of the Messaging API are listed below:

- Messages must only contain the text media type.
- Phone numbers must be in E164 format.

Requests and Responses

This chapter provides information about the requests and responses used in the Messaging API.

Constructing Requests

This section applies to all methods: `GET`, `POST`, `PUT`, and `DELETE`. Requests can be submitted for the following resources:

- Session: Used to create a unique and anonymous user session.
- Conversation: Used to manage conversations.

The API client needs to use the following headers in each request following an `/auth/anon` request:

- `Authorization`. Ensure that the token passed is prefixed with `Bearer-` (including the dash), as shown below:

  ```
  Authorization: Bearer-<token value>
  ```

- `farmId`.

Example

A request header with the session token and farm ID is shown below:

```
$ curl ... -H "Authorization: Bearer-<session token>" -H
"farmId: <farm ID>"
```

Each request contains secure HTTP headers and may contain parameters:

- Headers: `Content-type` in each request. Other headers are inserted automatically depending on your library. If the request contains a body (`POST` and `PUT` requests), include the `Content-type` header:

  ```
  Content-type:application/json; charset=utf-8
  ```

- Request data: Each request may contain data in these locations:
  - `Body`: The payload, JSON or multipart form data.
  - `Path`: Each resource has a unique URI that is built on the client side.
  - `Query`: Most parameters follow the URL in `GET` requests.

Syntax

This section provides information about the syntax to use when making API calls.

Use this syntax for all requests:
`https://<Base_API_URL>/<Context>/<Path>[?<Query_Parameter>...]`

Initiating the Login

To initiate the login process, use the common base URL for your region. For example, US: `https://app.five9.com:443`.

This URL handles the authentication process and returns the host data center information in the `apiUrls` used for all subsequent API requests. To construct the Base_API_URL, combine the host and port values from the `apiUrls` array. For example, the Base_API_URL would be: `https://app-scl.five9.com:443/`. For more information, see **Creates a Session for a Non-Agent User**.

Element	Description
`Base_API_URL`	Common base URL and port of the Five9 REST API server: • US: `https://app.five9.com:443` • UK: `https://app.five9.eu:443` • CA :`https://app.ca.five9.ca:443`
`Context`	Path to the Messaging REST API services: `/appsvcs/rs/svc`
`>Path`	Path to the request, such as `/auth/anon`
`Query_String`	Possible parameters depending on the request, such as `?cookieless=true`

Body

Some requests include a body. By default, the body is a JSON-formatted structure that contains required and optional parameters.

Responses

The HTTP response contains these elements:

- Success or failure HTTP code.
- Depending on the request, information in HTTP headers and body:
  `Content-type:application/json`

Status Codes

Requests can return these status codes:

Status Codes	Description
200	Request was successful.
202	Request was accepted for processing.
204	Request was successful.
401	User is not authorized to make the request.
403	User is not permitted to make the request.
429	Too many requests were sent to the server. Resend your request later.
500	An internal server error occurred during request processing.

Conversations

Use the Messaging API to execute these requests:

> **Important**
>
> The Messaging API supports UTF-8 character encoding. The number of characters specified for each field is not restricted. The value of the `Content-type` header should be `application/json; charset=utf-8`.

Creates a Session for a Non-Agent User

`POST /auth/anon`

Establishes an anonymous session for the chat client for users who are not agents.

> **Important**
>
> Follow the steps in this section before using the `/conversations` API.

Create a session by following these steps:

Create an Anonymous Session

Conversations are created asynchronously and specific to the domain and campaign. Each conversation must use a unique session. Because an existing session cannot be used to create or access multiple conversations, before starting a conversation, the API client must obtain an anonymous session.

> **Important**
>
> The anonymous session token will expire after eight hours of inactivity. If the session expires before the conversation finishes, the client application can no longer send messages to this conversation. If the client application maintains a long-running conversation with significant user inactivity intervals, it has to call /conversations/{conversationId}/info endpoint every seven hours to prevent session expiration.

Parameters.

Name	Data	Description	Type
`body`	`AnonymousUser`	Required context information about users who are not agents.	`Body`
`cookieless`	`boolean`	Required parameter that indicates whether or not authorization cookies should be created. Always set this parameter to true.	`Query`

Example

Creates an anonymous session.

```
$ curl https://<Base_API_URL/context>/auth/anon?cookieless=true --
data \ '{"tenantName": "tenant"}' -H "Content-type: applic-
ation/json" | jq
```

Response.

Type	Description
SessionToken	Authorization and authentication information received after a session has been created successfully.

Example

Response:

```
{  "tokenId": "fe2a5e5e-257d-11e9-aa60-005056b17082",  "orgId":
"100",  "userId": "fe2a5e5e-257d-11e9-aa60-005056b17082",  "con-
text": {    "farmId": "1"  },  "metadata": {    ...  }
}
```

Retrieve the following items from the response and pass them in subsequent requests:

- Farm ID in the `farmId` header.
- Session token in the Authorization header. Ensure that the token is prefixed with `Bearer-` (including the dash), as shown in the following example:

  Example

  Authorization: `Bearer-<token value>`

This means that for the `/auth/anon` API response example given above, the API request header would look like the following:

```
Authorization: Bearer-fe2a5e5e-257d-11e9-aa60-005056b17082
farmId: 1
```

Retrieve the Data Center Host Name

After creating a session and retrieving the `SessionToken` metadata, the client needs to obtain the name of the host to which requests need to be sent.

Example

Request:

Retrieves the host name of the data center.

```
$ curl -k https://<Base_API_URL/context>/auth/metadata -H "Author-
ization: Bearer-token"
```

Example

Response:


```
{"tokenId": "fe2a5e5e-257d-11e9-aa60-005056b17082",
"orgId": "100",
"userId": "fe2a5e5e-257d-11e9-aa60-005056b17082",
"context": {
"farmId": "1"
},
"metadata": {
"freedomUrl": "https://<host name>",
"dataCenters": [
{
"name": "Atlanta Data Center",
"uiUrls": [
{
"host": "<datacenter host name>",
"port": "443",
"routeKey": "ATLUIAkk1R",
"version": "12.0.22"
}
],
"apiUrls": [
{
"host": "<datacenter host name>",
"port": "443",
"routeKey": "ATLAPIsMPx",
"version": "12.0.22"
}
],
"loginUrls": [
{
"host": "<datacenter host name>",
"port": "443",
"routeKey": "ATLLGNPOE9",
"version": "12.0.22"
}
],
"active": true
}]}}
```


A sample response is shown below:

Retrieve the following items from the response and pass them in subsequent requests:

- Farm ID.
- Port number and data center host name from the `metadata.datacenters [@.active==true].apiUrls` object.

Important

If your domain has been migrated, you may see the following error message after making an API call:

```
Service migrated please retry your request
```

To resolve this issue, retrieve the domain's updated configuration before attempting to make the API call again. For more information, see **Domain Migration**.

Gets Session Metadata

`GET /auth/metadata`

Retrieves the sessions' metadata.

After creating a session, submit the `/auth/metadata` request. This request sends a response after the session information has been replicated across the cloud. Complete this step before calling the /conversations API.

Parameters.

Name	Data	Description	Type
`Authorization`	`string`	Required API token prefixed with the `Bearer-` string.	`Header`
`farmId`	`string`	Required farm ID.	`Header`

Example request

Retrieves the sessions' metadata.

`https://<Base_API_URL/context>/auth/metadata`

Response.

Type	Description
`SessionToken`	Authorization and authentication information received after a session has been created successfully.

Creates a Conversation

`POST /conversations`

Initiates a conversation between a customer and an agent.

The client must provide a callback URL for webhook notifications. After creating the conversation, the client must wait to receive the **Conversation Created** notification before submitting additional requests.

Parameters.

Name	Data	Description	Type
`Authorization`	`string`	Required API token prefixed with the `Bearer-` string.	`Header`
`body`	**`Conversation`**	Required conversation details.	`Body`
`farmId`	`string`	Required farm ID.	`Header`

Example

Creates a conversation.

`https://<Base_API_URL/context>/conversations`

Response.

Data	Description
`ConversationInfo`	Conversation details.

Gets Conversation Information

`GET /conversations/{conversationId}/info`

Retrieves conversation information.

Parameters.

Name	Data	Description	Type
`Authorization`	`string`	Required API token prefixed with the `Bearer-` string.	`Header`
`conversationId`	`string`	Required conversation ID.	`Path`
`farmId`	`string`	Required farm ID.	`Header`

Example

Returns information about conversation ID 1234.

```
https://<Base_API_URL/context>/conversations/{12345}/info
```

Response.

Data	Description
`ConversationInfo`	Information about the conversation status and a disposition code (if any).

Sends a Message

`POST /conversations/{conversationId}/messages`

Sends a message to the agents participating in the conversation.

Parameters.

Name	Data	Description	Type
`Authorization`	`string`	Required API token prefixed with the `Bearer-` string.	`Header`
`conversationId`	`string`	Required conversation ID.	`Path`
`body`	`Message`	Required message content.	`Body`
`farmId`	`string`	Required farm ID.	`Header`

Example

Sends a message for conversation ID 1234.

```
https://<Base_API_URL/context>/conversations/1234/messages
```

Response. Empty.

Sends Typing Event

`PUT /conversations/{conversationId}/messages/typing`

Notifies the agent about a user typing a message.

Parameters.

Name	Data	Description	Type
Authorization	string	Required API token prefixed with the Bearer- string.	Header
conversationId	string	Required conversation ID.	Path
farmId	string	Required farm ID.	Header

Example

For conversation ID 1234, sends an agent a notification about a customer typing a message.

```
https://<Base_API_URL/context>/conversations/1234/messages/typing
```

Response. Empty.

Acknowledges Message

```
PUT /conversations/{conversationId}/messages/acknowledge
```

Acknowledges delivery of messages sent by agents.

Parameters.

Name	Data	Description	Type
Authorization	string	Required API token prefixed with the Bearer- string.	Header
body	AcknowledgeMessages	Required list of messages to acknowledge.	Body
conversationId	string	Required conversation ID.	Path
farmId	string	Required farm ID.	Header

Example

Sends the customer an acknowledgment that the message for conversation ID 1234 has been sent to the agent.

```
https://<Base_API_URL/-
context>/conversations/1234/messages/acknowledge
```

Response. Empty.

Terminates Conversation

`POST /conversations/{conversationId}/terminate`

Ends a conversation.

The follow-on notification is**Conversation Terminated**. You can terminate only those conversations that are in the `ACTIVE` state.

Parameters.

Name	Data	Description	Type
`Authorization`	`string`	Required API token prefixed with the Bearer- string.	`Header`
`conversationId`	`string`	Required conversation ID.	`Path`
`farmId`	`string`	Required farm ID.	`Header`

Example

Terminates conversation ID 1234.

```
https://<Base_API_URL/context>/conversations/1234/terminate
```

Response. Empty.

Notification Events

After a conversation has been created with Messaging API (refer to the **Creates a Conversation** section), a webhook notification event is sent to the endpoint for each application event that takes place. The webhook resource returns a 200 or 204 HTTP status code for successful notifications. If the webhook resource replies with a code other than 2xx or does not reply within five seconds, the application attempts to re-deliver the notification one more time after 10 seconds. The webhook events below are required to be implemented.

Example

Notifies the client that a customer has created a conversation.

```
POST /conversations/95c0a733-406b-11e9-8816-005056b1792d/create
HTTP/1.1
Content-Length: 128
Content-Type: application/json; charset=UTF-8
Host: localhost:8082
Connection: Keep-Alive
User-Agent: Apache-HttpClient/4.5.6 (Java/1.8.0_192)
Accept-Encoding: gzip,deflate

{"externalId":"external-id-value","correlationId":"95c0a733-406b-
11e9-8816-005056b1792d","timestamp":"2019-03-06T23:57:24.247Z"}
```

Use these events:

Conversation Created

`POST /conversations/{conversationId}/create`

A customer initiated a conversation.

This notification should follow **Conversation Created**.

Parameters.

Name	Data	Description	Type
`body`	`ConversationEvent`	Required event details.	`Body`
`conversationId`	`string`	Required conversation ID.	`Path`

Response. Empty.

Conversation Accepted By Agent

`PUT /conversations/{conversationId}/accept`

An agent accepted the conversation.

Parameters.

Name	Data	Description	Type
`body`	`ConversationAcceptEvent`	Required event	`Body`

Name	Data	Description	Type
		details.	
conversationId	string	Required conversation ID.	Path

Response. Empty.

Agent Joined Conversation

`POST /conversations/{conversationId}/participant`

An agent joined the conversation.

Parameters.

Name	Data	Description	Type
body	ConversationParticipantEvent	Required event details.	Body
conversationId	string	Required conversation ID.	Path

Response. Empty.

Conversation Transferred to Another Agent

`PUT /conversations/{conversationId}/transferToAgent`

The specified conversation was transferred to another agent.

Parameters.

Name	Data	Description	Type
body	DataCenterInfo	Required event details.	Body
conversationId	string	Required conversation ID.	Path

Response. Empty.

Conversation Transferred to Group

`PUT /conversations/{conversationId}/transferToGroup`

The conversation was transferred to a group.

Parameters.

Name	Data	Description	Type
`body`	`ConversationTransferToGroupEvent`	Required event details.	`Body`
`conversationId`	`string`	Required conversation ID.	`Path`

Response. Empty.

Agent Left Conversation

`DELETE /conversations/{conversationId}/participant`

The agent left the conversation.

Parameters.

Name	Data	Description	Type
`body`	`ConversationParticipantEvent`	Required event details.	`Body`
`conversationId`	`string`	Required conversation ID.	`Path`

Response. Empty.

Agent Typing Message

`PUT /conversations/{conversationId}/typing`

The agent is typing a message in the conversation.

Parameters.

Name	Data	Description	Type
body	ConversationTypingEvent	Required event details.	Body
conversationId	string	Required conversation ID.	Path

Response. Empty.

Message Added

`POST /conversations/{conversationId}/message`

A message was added to the conversation.

Parameters.

Name	Data	Description	Type
body	ConversationMessageEvent	Required event details.	Body
conversationId	string	Required conversation ID.	Path

Response. Empty.

Conversation Terminated

`POST /conversations/{conversationId}/terminate`

The customer, agent, or system ended the specified conversation.

This notification follows Terminates Conversation.

Parameters.

Name	Data	Description	Type
body	ConversationTerminateEvent	Required event details.	Body

Name	Data	Description	Type
conversationId	string	Required conversation ID.	Path

Response. Empty.

Conversation processing error

`POST /conversations/{conversationId}/error`

An error occurred during request processing.

This event can apply to all parts of a conversation.

Parameters.

Name	Data	Description	Type
body	ConversationErrorEvent	Required event details.	Body
conversationId	string	Required conversation ID.	Path

Response. Empty.

Data Types

These data types describe data used by the API as follows:

- Data accepted by requests as parameters.
- Data returned in the payload of responses.
- Data passed to the client in the payload of events.

> **Important**
> Most dates and times are in the ISO-8601 format. Others are specified as needed.

AcknowledgeMessage

Information about a message to acknowledge.

Name	Type	Description
`type`	**`AcknowledgeType`**	Required message type. Possible values are listed below: • `DELIVERED` • `READ` • `FAILED`
`messageId`	`string`	Required message ID.

AcknowledgeMessages

Information about the messages to acknowledge.

Name	Type	Description
`messages`	**`AcknowledgeMessage`**`[]`	Required list of messages to acknowledge.

AcknowledgeType

Enumeration of acknowledgments types.

Description
`DELIVERED`: Message has been delivered.
`READ`: Message has been read.
`FAILED`: Message delivery has failed.

AnonymousUser

Information about logged-in users who are not agents.

Name	Type	Description
tenantName	string	Required domain name. Used to route chat sessions to the correct agents.

Contact

Information about the contact involved in the conversation.

Name	Type	Description
city	string	Contact's city.
company	string	Contact's company.
email	string	Contact's email address.
firstName	string	Contact's first name.
gender	string	Contact's gender.
lastName	string	Contact's last name.
number1	string	Contact's primary phone number in E164 format.
number2	string	Contact's secondary phone number in E164 format.
number3	string	Contact's tertiary phone number in E164 format.
socialAccountHandle	string	Contact's social account handle.
socialAccountName	string	Contact's social account name.
socialAccountImageUrl	string	Link to the contact's profile picture.
socialAccountProfileUrl	string	Link to the contact's social account.
state	string	Contact's residential state.
street	string	Contact's residential street name.
zip	string	Contact's residential zip code.

ContentType

Enumeration of supported content types.

Name	Type	Description
GENERIC	`String`	
SMS	`String`	
RCS	`String`	
FACEBOOK	`String`	
TWITTER	`String`	
WHATSAPP	`String`	
WECHAT	`String`	
QQMOBILE	`String`	
APPLE_BUSINESS_CHAT	`String`	
SKYPE	`String`	
SNAPCHAT	`String`	
VIBER	`String`	
LINE	`String`	
TELEGRAM	`String`	
KAKAO	`String`	

Conversation

Information about the conversation.

Name	Type	Description
`attributes`	`map <string, string>`	Optional conversation attributes, such as IVR variables. The values of attributes are redacted if their value matches the pattern of a data redaction rule.

Name	Type	Description
		Example: A data redaction rule is used to match credit card numbers that contain 16 consecutive digits. If the value of `externalId` is `<provider>-<server>-6394-1092582457863427501`, it is transformed into `<provider>-<server>-6394-1090000000000000000` because it contains 16 consecutive digits.
		Example: Use the pre-defined `Question` attribute to add a subject to queued and assigned messages.
`callbackUrl`	`string`	Required callback URL for webhook notifications.
`campaignId`	`long`	Campaign ID. Ignored if the `campaignName` parameter is used. This parameter is deprecated, use `campaignName` instead.
`campaignName`	`string`	Name of the campaign associated with the conversation being created. This property takes priority over the `campaignId` parameter's value.
`contact`	`Contact`	Required information about the contact who requested the conversation.
`disableAutoClose`	`boolean`	If the auto close feature is enabled on the Text Channels Administrator Console, the message window automatically closes when there is no response from the customer's

Name	Type	Description
		side for a time period defined by the administrator. While using the Messaging API, the administrator can override this behavior by using the `disableAutoClose` parameter to prevent the conversation from closing automatically due to inactivity from the customer's side. The default value of this parameter is false.
`disableComfortMessage`	`boolean`	Whether to prevent a chat message from being sent to a customer if the agent has not responded for a timespan defined by the administrator at the profile level. This default value of this parameter is True.
`externalId`	`string`	Optional conversation ID in the third party application. This parameter is stored as a custom field and can be used as a CAV for Engagement Workflow or connectors.
`priority`	`integer`	Conversation priority in `int32` format.
`tenantId`	`long`	Required domain ID.
`type`	**`ContentType`**	Conversation content type.
`useBusinessHours`	`boolean`	Whether the campaign's business hours settings, defined in the Digital Engagement Administrator console, should be applied to conversations initiated by the customer. The default value of this parameter is false, with chat always open. When set to true, chats are routed normally during open business hours when agents are logged in and dropped when no agents are logged in. Chat

Name	Type	Description
		interactions received during closed business hours remain in the assigned queue for 24 hours.

ConversationAcceptEvent

Information about an accepted conversation.

Name	Data	Description
`correlationId`	`string`	Conversation ID.
`displayName`	`string`	Required agent's display name.
`externalId`	`string`	Optional conversation ID in the third party application. This parameter is stored as a custom field and can be used as a CAV for Engagement Workflow or connectors.
`eventSerialNumber`	`long`	Event serial number.
`from`	`ConversationBind`	Event source.
`timestamp`	`string`	Date and time of the event creation.
`userId`	`long`	Required agent ID.

ConversationErrorEvent

Information about the agent who joined or left the conversation.

Name	Data	Description
`correlationId`	`string`	Correlation ID of the conversation.

Name	Data	Description
contentType	ContentType	Content type.
displayName	string	Required agent's display name.
externalId	string	Optional conversation ID in the third party application. This parameter is stored as a custom field and can be used as a CAV for Engagement Workflow or connectors.
eventSerialNumber	long	Event serial number.
from	ConversationBind[]	List of supported event origins: SYSTEM or USER.
messageId	string	Message ID.
messageSerialNumber	long	Message serial number.
messageType	long	Message type.
text	string	Message content.
timestamp	string	Date and time of the event creation.
userId	long	Required agent ID.

ConversationEvent

Information about the conversation to be created.

Name	Data	Description
correlationId	string	Conversation ID.
externalId	string	Optional conversation ID in the third party application. This parameter is stored as a custom field and can be used as a CAV for Engagement Workflow or connectors.
timestamp	string	Date and time of event creation.
eventSerialNumber	long	Event serial number.

ConversationEventOrigin

Enumeration of sources that can generate events.

Description
`SYSTEM`: Enumeration of source systems.
`USER`: User for whom the event was generated.

ConversationInfo

Information about the conversation.

Name	Type	Description
`id`	`string`	Conversation ID.
`status`	**`ConversationTypingEvent`**	Enumeration of conversation status types. Possible values are listed below: • `PENDING` • `ACTIVE` • `TERMINATED`
`disposition`	`long`	Conversation's disposition code.

ConversationMessageEvent

Information about the message added to the conversation.

Name	Data	Description
`contentType`	**`ContentType`**	Content type.
`correlationId`	`string`	Correlation ID of the conversation.
`displayName`	`string`	Required display name of the agent.

Name	Data	Description
eventSerialNumber	long	Event serial number.
externalId	string	Optional conversation ID in the third party application. This parameter is stored as a custom field and can be used as a CAV for Engagement Workflow or connectors.
from	string	Event source.
messageId	string	Message ID.
text	string	Optional message content.
timestamp	string	Date and time of event creation.
userId	long	Agent ID.
attachments	[string]	Array of file identifiers issued by Files API.

ConversationParticipantEvent

Information about the agent who joined or left the conversation.

Name	Data	Description
correlationId	string	Correlation ID of the conversation.
displayName	string	Agent's display name.
eventSerialNumber	long	Event serial number.
externalId	string	Optional conversation ID in the third party application. This parameter is stored as a custom field and can be used as a CAV for Engagement Workflow or connectors.
participants	Metadata <not in swagger>[]	Required list of agents participating in the conversation.
timestamp	string	Date and time of event creation.
userId	long	Required agent ID.

ConversationStatus

Enumeration of conversation states.

Description
`ACTIVE`: Conversation is active.
`PENDING`: Conversation creation is pending.
`TERMINATED`: Conversation has ended.

ConversationTerminateEvent

Information about conversation termination.

Name	Data	Description
`correlationId`	`string`	Conversation ID.
`displayName`	`string`	Required agent's display name.
`externalId`	`string`	Conversation’s external ID, which can be used for storing in an external application. This parameter is stored as a custom field and can be used as a CAV for Engagement Workflow or connectors.
`eventSerialNumber`	`long`	Event serial number.
`from`	`ConversationBind`	Event origin.
`timestamp`	`string`	Date and time of event creation.
`userId`	`long`	Required agent ID.

ConversationTransferEvent

Information about a conversation transferred to another agent.

Name	Data	Description
correlationId	string	Conversation ID.
externalId	string	Conversation's external ID, which can be used for storing in an external application. This parameter is stored as a custom field and can be used as a CAV for Engagement Workflow or connectors.
eventSerialNumber	long	Serial number of the event.
fromDisplayName	string	Required name of the agent who transferred the conversation.
fromUserId	long	Required ID of the agent who transferred the conversation.
timestamp	string	Date and time of event creation.

ConversationTransferToAgentEvent

Information about a conversation transferred to another agent.

Name	Data	Description
correlationId	string	Correlation ID of the conversation.
eventSerialNumber	integer	Event serial number in int64 format.
externalId	string	Optional conversation ID in the third party application. This parameter is stored as a custom field and can be used as a Call Attached Variable for Engagement Workflow or connectors.
fromDisplayName	string	Required name of the agent who transferred the conversation.
fromUserId	long	Required ID of the agent who transferred the conversation.
participants	Metadata <not in swagger>[]	List of participating agents.
timestamp	string	Required date and time of event creation.

Name	Data	Description
toDisplayName	string	Required display name of the agent to whom the conversation was transferred.
toUserId	long	Required ID of the agent to whom the conversation was transferred.

ConversationTransferToGroupEvent

Information about a conversation transferred to a group.

Name	Data	Description
correlationId	string	Correlation ID of the conversation.
eventSerialNumber	long	Event serial number.
externalId	string	Optional conversation ID in the third party application. This parameter is stored as a custom field and can be used as a Call Attached Variable for Engagement Workflow or connectors.
fromDisplayName	string	Required ID of the agent who transferred the conversation.
fromUserId	long	Required ID in of the agent who transferred the conversation.
groupId	long	Required group ID.
groupName	string	Required display name of the group.
timestamp	string	Date and time of the event creation.

ConversationTypingEvent

Information about the agent typing a message.

Name	Data	Description
correlationId	string	Conversation ID.
displayName	string	Conversation's external ID, which can be used for storing in an external application. This parameter is stored as a custom field and can be used as a Call Attached Variable for Engagement Workflow or connectors.
externalId	string	Optional conversation ID in the third party application. This parameter is stored as a custom field and can be used as a Call Attached Variable for Engagement Workflow or connectors.
eventSerialNumber	long	Event serial number.
timestamp	string	Date and time of the event creation.
userId	long	Required agent ID.

Generic500ErrorResponse

Information about the error.

Name	Type	Description
description	string	Error description.

Group

Information about a group.

Name	Type	Description
groupId	long	Required group ID.
groupName	string	Required group display name.

Message

Information about a message.

Name	Type	Description
`messageType`	`MessageType`	Message type.
`message`	`string`	Required content of the conversation message.
`externalId`	`string`	Optional conversation's Id in the third party application.
`attachments`	`[ string ]`	Array of file identifiers issued by Files API.

MessageType

Enumeration of the possible types of messages.

Description
`TEXT`: Text message.

Participant

Information about a conversation participant.

Name	Type	Description
`userId`	`long`	Required user ID.
`displayName`	`string`	Required agent name.

SessionToken

Information about the authorization and authentication token received after a successful login.

Name	Type	Description
`context`	`Map<string, string>`	Optional session context information.
`metadata`	`metadata`	Information about the data center and host name.
`orgId`	`string`	Domain ID.
`tokenId`	`string`	Token ID.
`userId`	`string`	User ID.

Using the Messaging API Script

The Messaging API script can be used as a tool to get started with interacting with the Messaging API.

> **Note**
>
> The following procedure requires using `ngrok` or a similar service for performing callbacks. To use `ngrok`, please contact Five9 Support for guidance. You must have the `jq` command-line tool installed to perform this procedure.

1. Create a file, such as `messagingAPIScript.sh`.

   ```
   $ cat messagingAPIScript.sh
   ```

2. Add the following contents to the file.

```
################################################################
#
# Sample script for testing the Messaging API.
# Required tools - curl, jq
# Use https://ngrok.com/, pipedream.net, or a similar service
for
# callbacks
################################################################
#
#define variables
export five9Host='app.five9.com'
export tenantName='<your-tenant-name>'
export campaignName='<your-campaing-name>'
export phoneNumber='+15555550100'
export callbackUrl='https://<your-host>'

#get anon token
authInfo=`curl "https://$five9Host/appsvcs/rs/svc/ \
auth/anon?cookieless=true" \
  -H 'Content-Type: application/json'  -H 'Accept: \
application/json, text/javascript, */*; q=0.01'  \
  --data-binary "{\"tenantName\":\"$tenantName\"}"  | jq .`

token=`echo $authInfo | jq -r '.tokenId'`
metaHost=`echo $authInfo | jq -r '.metadata.dataCenters
[0].apiUrls[0].host'`
```

```
metadataInfo=`curl "https://$metaHost/appsvcs/rs/svc/ \
auth/metadata" \
  -H "Authorization: Bearer-$token" \
  -H 'Accept: application/json, text/javascript, */*; q=0.01'
\
| jq . `

targetHost=`echo $metadataInfo | jq -r '.metadata.dataCenters
[0].apiUrls[0].host'`
orgId=`echo $metadataInfo | jq -r '.orgId'`
farmId=`echo $metadataInfo | jq -r '.context.farmId'`

conversation=`curl -X POST "https://$targetHost/appsvcs/rs/ \
svc/conversations" \
 -H "Authorization: Bearer-$token" \
 -H "accept: application/json" \
 -H "farmId: $farmId"  -H "Content-Type: application/json" \
 -d "{\"tenantId\":$orgId, \"campaignName\":
\"$campaignName\",
    \"type\":\"SMS\",
      \"contact\":{\"number1\":\"$phoneNumber\"},
      \"priority\":1,
      \"callbackUrl\":\"$callbackUrl\",
      \"attributes\":{ \"Question\":\"I need help\"} }" | jq .
`

conversationId=`echo $conversation | jq -r '.id'`

echo Conversation ID is $conversationId

#Wait 1s for conversation creation.
sleep 1s

#Send a message
curl -X POST "https://$targetHost/appsvcs/rs/svc/ \
conversations/$conversationId/messages" \
 -H "Authorization: Bearer-$token" \
 -H "accept: application/json" \
 -H "farmId: $farmId"  -H "Content-Type: application/json" \
 -d "{\"messageType\": \"TEXT\", \"message\": \"Hello ..\"}"
Save and file.
Execute the script to create a conversation object.
$ messagingAPIScript.sh
```

Engagement Workflow Customization

Call variables are custom fields that store data. Agents can use and update these call variables in the IVR during calls. When configuring a disposition, you can also set the value of the call variables that should be stored for reporting purposes when that disposition is used. With call variables, you can gather additional information about each call. You can also use preconfigured custom call variables to customize wait messages.

> **Important**
>
> Customizing wait messages with the Engagement Workflow applies only to chats that have been created with the Messaging API.

To learn more about configuring call variables, refer to the Interactive Voice Response (IVR) Administrator's Guide and Digital Engagement Administrator's Guide.

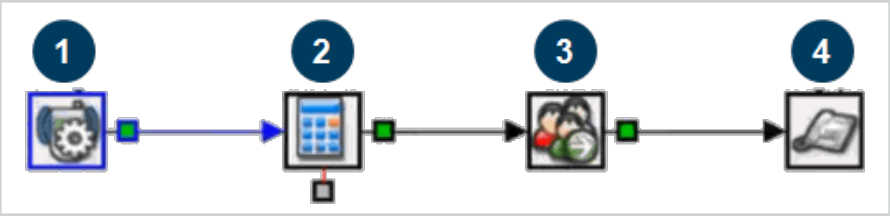
Custom Call Variables Used by the Messaging API

The Messaging API uses the following custom call variables:

Name	Description
`Custom.dynamic_wait_message`	Sent when ACD is able to determine wait times.
`Custom.disableAutoClose`	Created if the value of the attribute `disableAutoClose` is `true`.
`Custom.disableComfortMessage`	Created if the value of the attribute `disableComfortMessage` is `true`.
`Custom.externalId`	Pass-through ID that refers to the conversation object in an external system.
`Custom.no_service_message`	Sent when there are no logged-in agents.
`Custom.submedia_type`	Indicates the source of the message, such as Facebook or Twitter.
`Custom.wait_message`	Generic wait messages.

Customizing Wait Messages

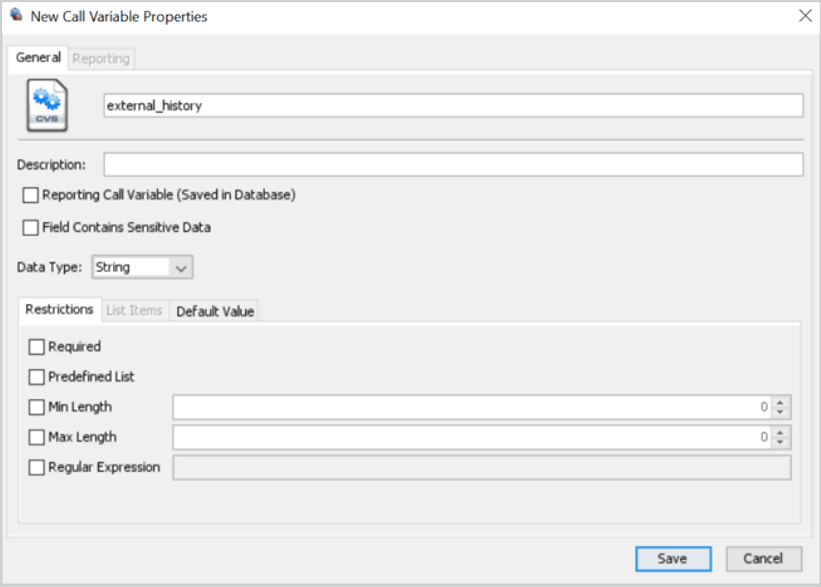
The workflow of customizing wait messages with Engagement Workflow is shown in the following image:



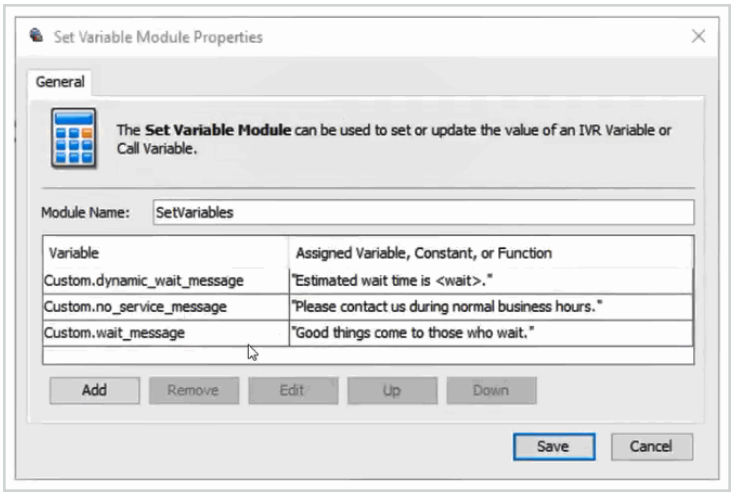
1 **Incoming Call** module
2 **Set Variables** module – Set variable values.
3 **Skill Transfer** module – Skill transfer to push variable values back to the system
4 **Hangup** module

Use the following steps to customize a wait message:

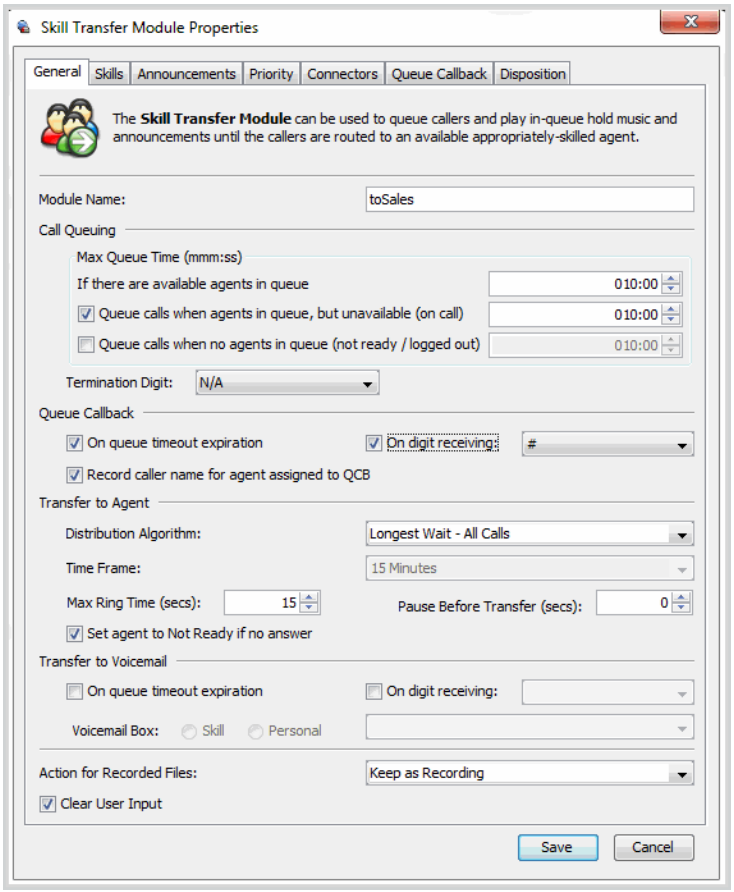
1 Configure a custom variable in the VCC Administrator application or use a preconfigured custom variable.



2 Use the Set Variable module to assign a value to the variable.



3 Configure the Skill Transfer module in the VCC Administrator application.



For detailed information about module configuration, see the Interactive Voice Response (IVR) Administrator's Guide.

Using the Messaging API With a Connector

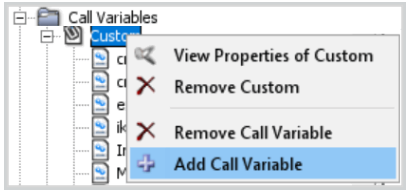
This procedure provides instructions for using the Messaging API with a connector to perform various tasks, such as viewing contact information.

Perform the following steps in the order listed:

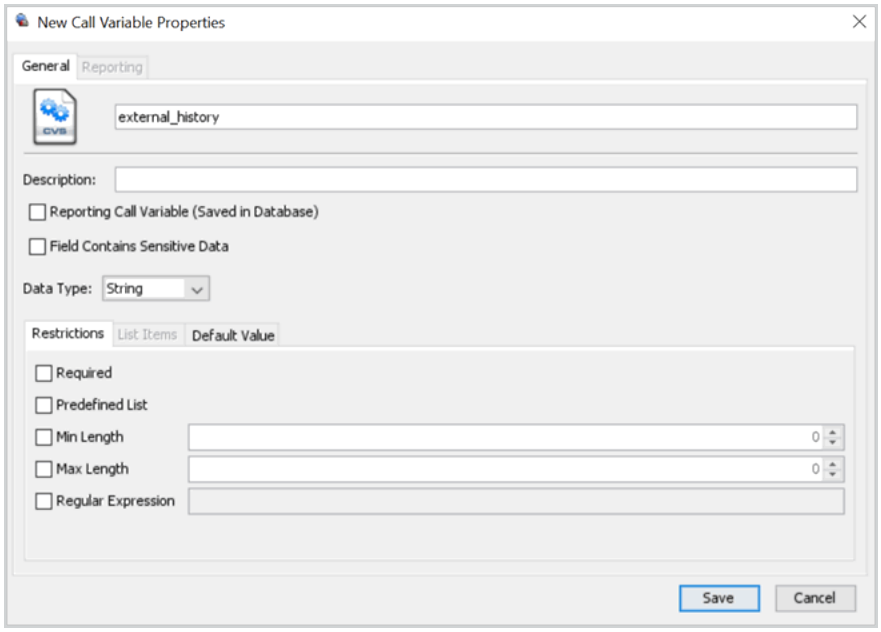
Configuring a Custom Variable and Connector

Perform the following configuration steps in the VCC Administrator Application.

1. Log into the VCC Administrator application as an administrator.
2. Create a new custom variable.
   a. Expand **Call Variables** and select locate the **Custom** group.
   b. Right-click **Custom** and select **Add Call Variable**.



   c. Enter a name for the variable, such as `external_history`.

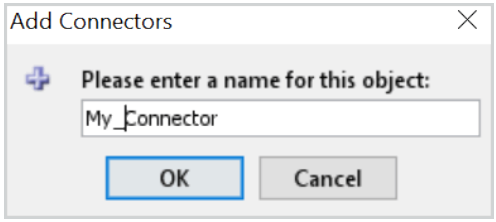


d Click **Save**.

3 Create a new connector.

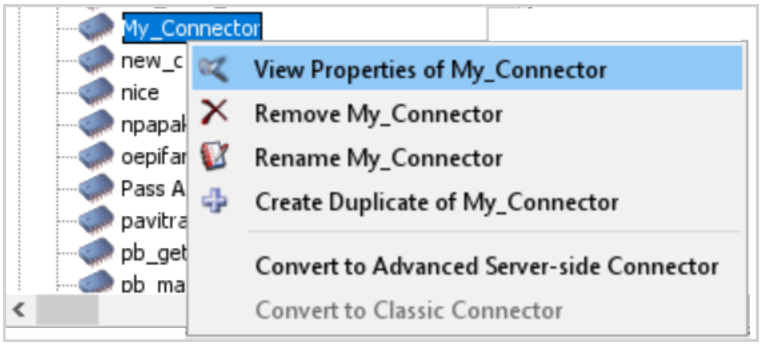
a Right-click **Connectors** and select **Add Connector**.

b Enter a name for the connector.



c Select **Classic Connector** as the type and click **Create Connector**.

d Optionally, right-click the new connector to view properties.



4 Configure the connector.

a Click the **General** tab.

b Optionally, enter a description for the connector.

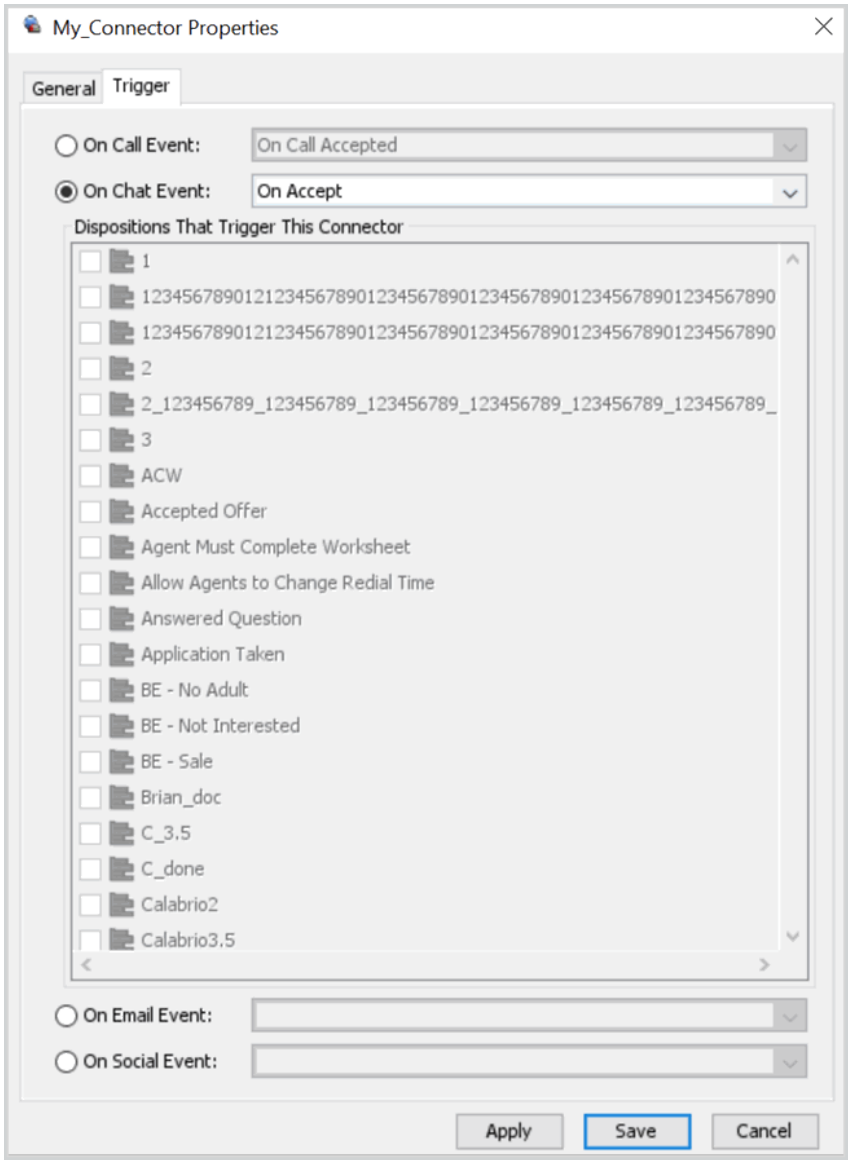
c Enter the URL of the site to which you want to integrate the connector.

d Click **Add Field** and select the newly created variable.

The **Parameters** section is used for mappings context variables to external parameters.

e Select **In Browser** in the **Execution Mode** section.

f Click the **Trigger tab** and select **On Chat Event** to specify the desired connector activation trigger event.

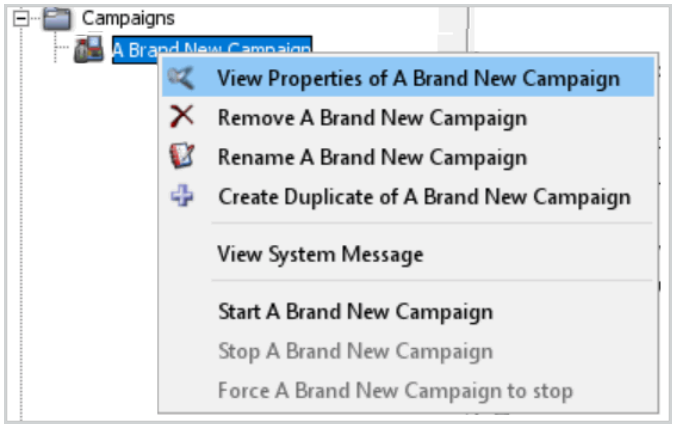


g Click **Save.**

5 Associate the new connector with the desired campaign.

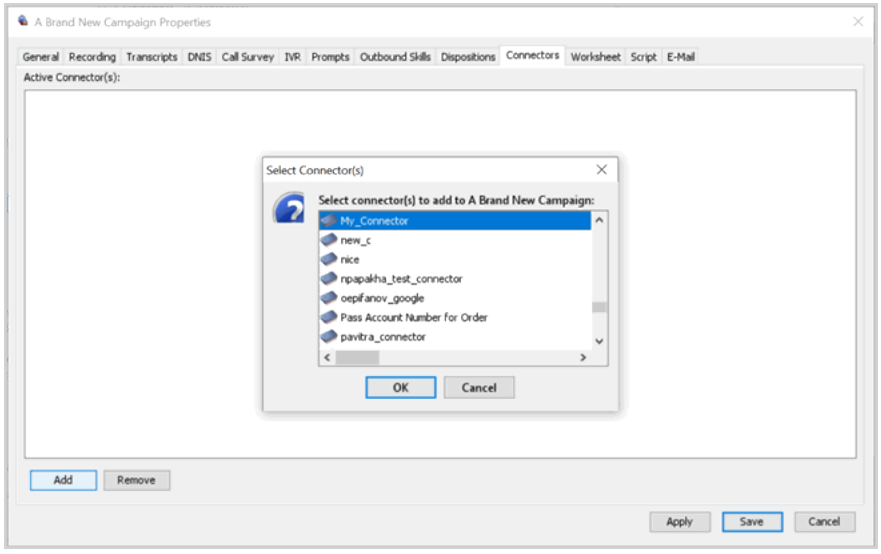
a Expand **Campaigns** and locate the desired campaign.

b Optionally, right-click the campaign to view properties.



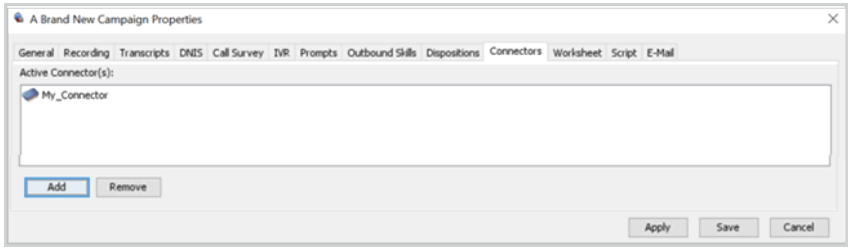
c Select the **Connectors** tab and click **Add**.

d Select the new connector and click **OK**.



The new connector appears in the list of active connectors.

e Click **Save**.



Creating a Conversation

Create a conversation with the Messaging API by following these steps:

1. Get authenticated. For more information, refer to **Creates a Session for a Non-Agent User**.
2. Create a new conversation. In the request body, pass the custom variable created in **Creates a Conversation** as an attribute of the **Conversation** object.

Example

Request URL:

```
POST /conversations
```

Example

Request body:

```
{  "campaignName": "My_Campaign",
"tenantId": 12345,
"externalId": "123",
"type": "RCS",
"contact": {
"state": "CA",
"zip": "94568",
"city": "Pleasanton",
"street": "ABC",
"company": "Demo Company",
"firstName": "John",
"lastName": "Smith",
"gender": "Male",
"email": "abc@xyz.com",
"number1": "123",
"number2": "456",
"number3": "789",
"socialAccountHandle": "A",
"socialAccountName": "B",
"socialAccountImageUrl": "URLA”,
"socialAccountProfileUrl": "URLB"
},
"priority": 12345,
"callbackUrl": "URLC",
"attributes": {
"question": "How are you?",
"Custom.external_history": "Hello"
}
}
```

In the preceding example, it is assumed that the name of the custom variable is `external_history`.

Testing the Integration

Use these instructions to verify that the Messaging API has been integrated correctly.

1 Send a test **Message** by using the Messaging API.

Example

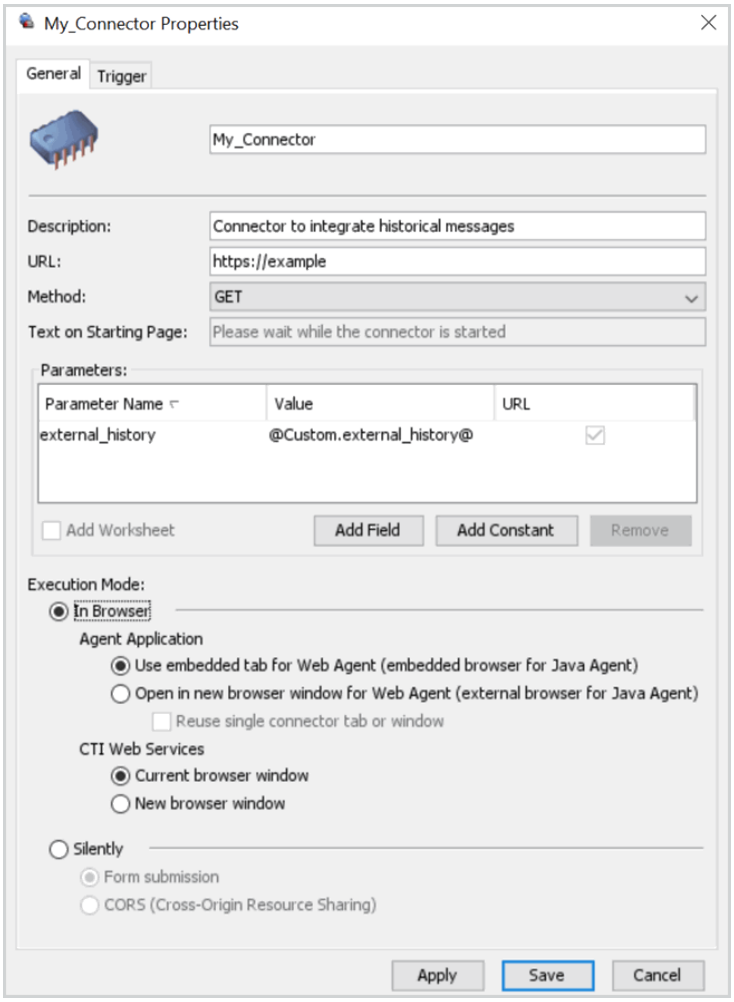
Request URL:

```
POST /conversations/1234/messages
```

2 Log into ADP as an agent and accept the conversation.

A new browser window is opened, displaying the URL that was specified while configuring the connector. You can change how the browser window is displayed

by specifying the desired settings in the connector's configuration.



Troubleshooting Information

Use the information in this chapter to troubleshoot common issues encountered while using the Messaging API.

Events and Notifications

Webhook notification events are sent to endpoints for application events. Use these guidelines to troubleshoot issues related to events and notifications.

Maintenance Events

During maintenance events, conversation processing may be moved between data centers. After maintenance is performed, the value of the `farmId` header becomes obsolete and the latest value of `farmId` returned by the metadata request should be used.

Important

The Messaging REST API does not support a special webhook for maintenance events. Instead, maintenance-related errors should be handled by the client applications.

Webhooks

If you notice that webhook notifications are not being delivered, ensure that the following guidelines are met:

- Do not use self-signed certificates.
- If the callback uses `ngrock.io` or a similar service, configure your proxy service to use a stable DNS domain name because the Five9 network does not allow connection to an anonymous proxy. Submit a request to Five9 Support for enabling connection to your domain.

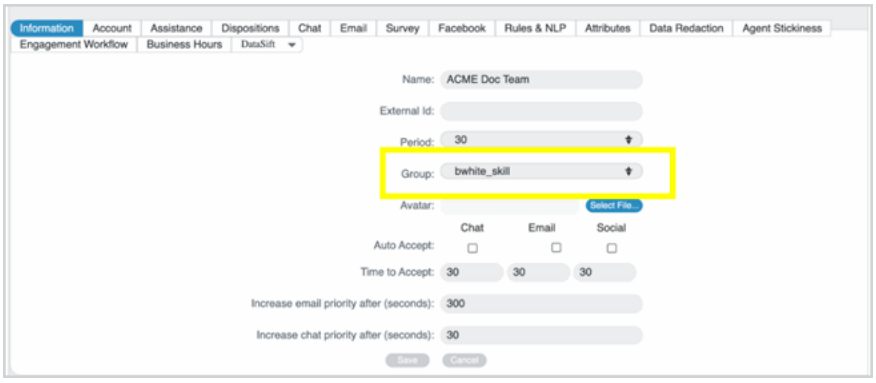
Miscellaneous Issues

Use these guidelines to resolve additional Messaging API issues.

Authentication

If you run into issues related to authentication, ensure that the following guidelines are met while submitting requests:

- The tenant and campaign names are case sensitive. Ensure that they are entered correctly.
- Ensure that the campaign profile has an associated group defined in the Digital Engagement Administrator application. For more information, refer to the Digital Engagement Administrator's Guide.



- Ensure that the `farmId` header is provided for each request.
- Use a unique anonymous session token for each conversation.

Business Hours

The `useBusinessHours` parameter indicates whether the campaign's business hours setting should be applied to conversations initiated by the customer. The default value of this parameter is `false`. To prevent issues related to business hour messages, ensure that the following guidelines are met:

- If the value of the `useBusinessHours` is parameter is set to `false`, ensure that the agents are assigned a skill which is mapped to the corresponding profile used in the Digital Engagement Administrator application.
- Ensure that the agent's skill is enabled for chat channels.
- Ensure that the business hour setting is configured for the profile in the Digital Engagement Administrator application.

Domain Migration

Making an API call after your domain has been migrated to a different data center may result in the following error message:

```
Service migrated please retry your request
```

Follow these steps to resolve this issue:

1. **Retrieve the domain's updated configuration.**

   Example

   ```
   $ curl -k https://<Base_API_URL/context>/auth/metadata -H \
   "Authorization: Bearer-token"
   ```

2. Retrieve the domain's configuration again by using the `farmID` retrieved in the previous step and the base URL from the `metadata.datacenters [@.active==true].apiUrls` object.

   Example

   ```
   $ curl -k https://<Base_API_URL/context>/auth/metadata -H \
   "Authorization: Bearer-<token>" -H "farmId: <farmId>"
   ```

   The server responds with the new metadata after the authentication context has been fully replicated on the data center.

3. Retry the API call with the retrieved `farmId` and host name.

Using the Logged In Profiles Endpoint for Agent Availability Check

Use the `logged_in_profiles` endpoint to verify if any agents are logged in for a given profile (campaign).

Retrieve detailed information about the campaigns in the list.

Access this endpoint with the token obtained from the `/auth/anon` endpoint. This provides details on whether an agent is logged in for a specific campaign (true or false).

Availability Check Work Flow

Follow the steps to use `/logged_in_profiles` within your work flow:

1. Authenticate: Call `/auth/anon?cookies=true`. Start by calling the `/auth/anon` endpoint with the cookies=true parameter to initiate an anonymous login. This returns an authentication token that you'll use for subsequent requests.
2. Get Metadata: Call `/auth/metadata`. Retrieve the required host and port information by calling `/auth/metadata`.
3. Check Agent Availability: Call `/logged_in_profiles`. Use the `/logged_in_profiles` endpoint to check if agents are logged in for your specified campaigns. Define a list of profiles (such as specific campaigns) and check their availability. This enables you to make informed decisions about whether to proceed with initiating a conversation.

Endpoint

```
GET /appsvcs/rs/svc/agents/{tokenId}/logged_in_profiles?profiles=lc_chat,lc_marketing
```

Path Parameter

Parameter	Type	Description
tokenId	String	Your token identifier.

Query Parameter

Parameter	Type	Required	Description
profiles	String	Yes	A comma-separated list of campaign names to check availability.

Example Response

```
[
    {
        "profileId": "1552343965",
        "profileName": "lc_chat",
        "agentLoggedIn": false,
        "noServiceMessage": "We are currently unable to service your
request. Please contact us during normal business hours.",
        "emailRequired": true,
        "restrictions": {
            "maxNameLength": 64,
            "maxEmailLength": 128,
            "maxQuestionLength": 255
        },
        "enableCustomerRequestChatTranscript": false,
        "chatbotEnabled": false,
        "enableRoutingAfterBusinessHours": false,
        "enableRoutingWhenNoAgentsAreLoggedIn": false,
        "openForBusiness": true
    },
    {
        "profileId": "1600012723",
        "profileName": "lc_marketing",
        "agentLoggedIn": false,
        "noServiceMessage": "We are currently unable to service your
request. Please contact us during normal business hours.",
        "emailRequired": true,
        "restrictions": {
            "maxNameLength": 64,
            "maxEmailLength": 128,
            "maxQuestionLength": 255
        },
        "enableCustomerRequestChatTranscript": false,
        "chatbotEnabled": false,
        "enableRoutingAfterBusinessHours": false,
        "enableRoutingWhenNoAgentsAreLoggedIn": false,
        "openForBusiness": true
    }
]
```